DISTRIBUTED COMPUTING ANALYSIS REPORT

Load Rebalancing For Hadoop Distributed File System Using Distributed Hash Table

M. Nithya, N. Uma Maheshwari

Department Of Computer Science and Engineering, PSNA College Of Engineering and Technology, Dindigul, India

2017 International Conference on Intelligent Sustainable Systems (ICISS), Palladam, *2017, pp. 939-943.*

Team Members:

Name	Roll No	Email ID
Anuj Revankar	15CO109	anujrevankar@gmail.com
Kiran S	15CO125	krnsnaik@gmail.com
Prateek Kembhavi	15CO223	prateekkembhavi@gmail.com

Abstract

In the growing world of networking, it is very important to have a system where the files are distributed among a network of servers instead of a centralized structure. Apache Hadoop is a collection of open-source software utilities that facilitate using a network of many computers to solve problems involving massive amounts of data and computation. It provides a software framework for distributed storage and processing of big data using the properties of Google's File system and map reduce. Even though the system in general is distributed, but there is one major issue with the system. Every transaction in the Hadoop file system has to move through the master node which provides the address of the required data node. This creates a centralized structure at the master node which becomes a bottleneck in heavy load conditions. Secondly, the system become fault intolerant at the master node. To address this problem, in the paper, we have proposed to Distributed hash table at the data nodes to find the address of the required data nodes. The algorithm used in the paper also balances itself according to the load such that any node doesn't get overflowed. As every node is involved in delivering the address, it is also important to ensure security with each node. In our case, we have used AES to secure each of the nodes.

Problems addressed in the paper

Hadoop's distributed file system works on Google Seminar paper "Google file system". In this system, data is stored in different distributed systems in the form of nodes. There are two types of nodes namely:

- **Master Node**: This node saves the location of the data stored in the distributed file system. Along with this, it stores the metadata of all the files like last updated, read-write permissions, etc.
- **Data Nodes**: These Nodes hold the actual data that needs to be stored in the file system.

This paper discusses the centralized structure of the master node.

The problems are as follows:

- Every transaction on the file system must pass through the master node, to find its address and the necessary details. This creates a bottleneck problem (The load on the master node increases with the increasing frequency of transactions).
- As the master node is centralized, it becomes a victim to single point of failure as its failure makes the system inconsistent.
- As there can be a lot of load on the master node, it can cause crashing or corruption of data in the master node.

This paper solves the above problem with **Distributed hash table** which decentralizes the master node.

Solutions Proposed in the Paper

The main objective of the paper is to decentralize the master node (completely remove the master node) and implement a distributed hash table in all the data nodes The solution is proposed in 2 stages:

Uploading stage: This stage involves 3 sub stages.

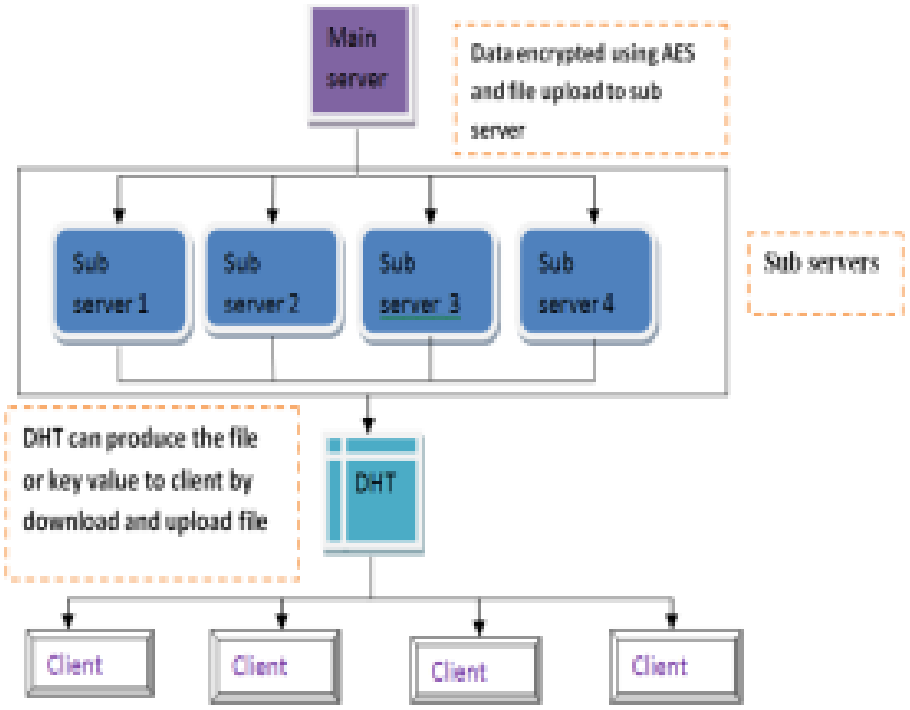
- **Chunk Creation**: In this stage, the data to be stored in the file system is divided in small chucks of data. The size of each block depends on the size of the whole file system.
- **DHT Formation**: In this stage, the data (divided in chunks) is given a key value using a hash function. A new entry is made in all the data nodes for the given chunk.
- **Resource Allocation**: In this stage, the free block is allocated to the chuck of data and its value (key) is updated in all the data nodes.

Downloading stage: The replica management works the same way as in the proposed in the original google file system paper. In this stage, client requests for a data chunk to the nearest data node. If the requested data node has the address for the required data, it provides the same. Else it provides the nearest data node that might have the requested data. Hence, we, move from node to node until we reach the required node.

Advantages of the proposed solution:

- There is no single point of failure
- The load on the master node is balanced through all the nodes
- The chances of crashes or corruption of data is minimized
- The system is decentralized completely

Proposed Solution



Issues addressed in your proposed solution

- As the system is distributed now, all the data nodes must have a reserved space for the data nodes (This is **not recommended** as it requires a whole change in the architecture)
- The computational time to find the data is increased. In the existing system the time complexity is **0(1)** while in this system it is **0(log(n))** where n is the total number of nodes
- Any corruption of data nodes will require the fields of all the nodes to be updated
- There is a major issue with this implementation, which is **processing continuity**. As we know, gaining the meta-data takes very less time as compared to data processing in the map-reduce phase. As the name node is absent, if any data-node is in use, it cannot be used to address the name node capability. In the case of large system, this might become a common problem and we will see drops in requests because the data nodes are busy with the map-reduce and can't handle the addressing at same time
- Another issue with this approach is **security**. As the name node are absent, the data nodes require extra security. The system becomes more vulnerable to attack and as the data and the indexes are present in the same nodes, we can see that data breaches are easier.

Our Solution

Summing up the problems in the proposed solution:

- Security
- Change in architecture of **HDFS** and **Map-Reduce**
- Not scalable
- Inefficient Performance

Considering the following issues with the proposed solution, we have proposed the **Multi-Name node** solution

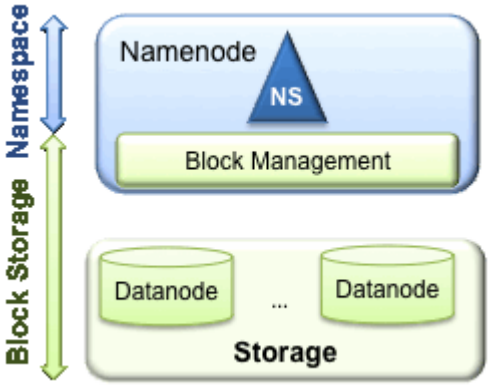
Multi-Namenodes Approach

- In this approach, we create multi name nodes instead of merging them with the data nodes themselves
- The number of name nodes depends on the number of data nodes
- The name nodes are independent and do not require coordination with each other.
- The Data nodes are used as common storage for blocks by all the Namenodes.
- Each Data node registers with all the Namenodes in the cluster. Data nodes send periodic heartbeats and block reports.
- They also handle commands from the Namenodes.
- The balancer in Hadoop is already updated to handle multiple name nodes
- Hadoop also has the capacity of creating new data nodes as per need in the newest update

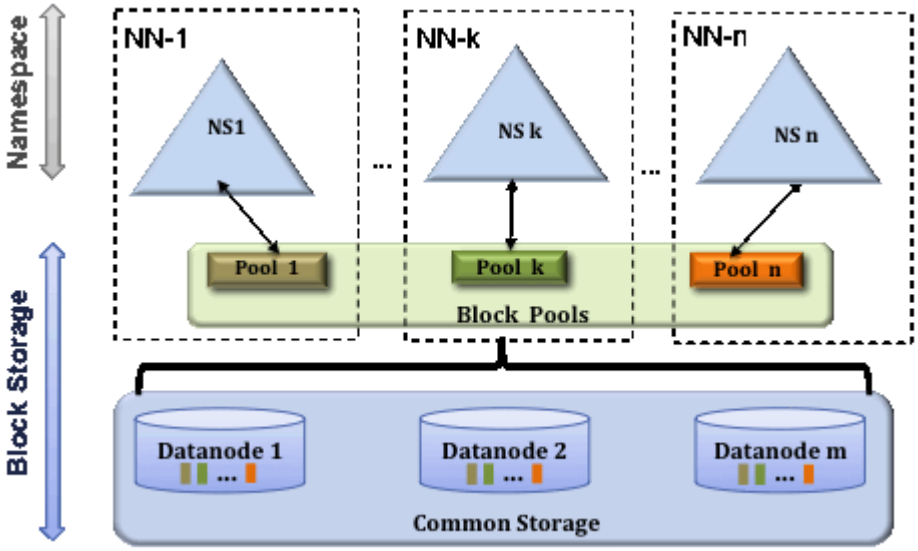
Advantages of this approach

- No extra security is required
- Architecture change only in HDFS part
- Performance is not compromised

Original System



Proposed System



Our Implementation

Here is the example code for configuration for 2 Name nodes:

```
<configuration>
  <property>
    <name>dfs.nameservices</name>
    <value>ns1,ns2</value>
  </property>
  <property>
    <name>dfs.namenode.rpc-address.ns1</name>
    <value>nn-host1:rpc-port</value>
  </property>
  <property>
    <name>dfs.namenode.http-address.ns1</name>
    <value>nn-host1:http-port</value>
  </property>
  <property>
    <name>dfs.namenode.secondaryhttp-address.ns1</name>
    <value>snn-host1:http-port</value>
  </property>
  <property>
    <name>dfs.namenode.rpc-address.ns2</name>
    <value>nn-host2:rpc-port</value>
  </property>
  <property>
    <name>dfs.namenode.http-address.ns2</name>
    <value>nn-host2:http-port</value>
  </property>
  <property>
    <name>dfs.namenode.secondaryhttp-address.ns2</name>
    <value>snn-host2:http-port</value>
  </property>

  .... Other common configuration ...
</configuration>
```

Conclusions

The proposed system put forth the Load Rebalancing in each block in the Hadoop cluster and data reconstruction system. For block failure, issues on Hadoop cluster are overcome. For the reconstruction process we apply multiple names over the same Hadoop cluster. Finally, the multiple name nodes are configured to work with the balancer which considers the functioning of the map-reduce and the hbase in the Hadoop cluster. Data can once again be stored inside the cloud but the data has to be divided into small pieces. For this process we create separate name nodes and each name nodes independently function and know the existance of the other name nodes and hence hence help in sharing the load

References

J Dean, S Ghemawat, "Mapreduce: simplified data processing on large clusters", Communications of the ACM, vol. 51, no. 1, pp. 107-113, 2008.

S. Ghemawat, H. Gobioff, S.T. Leung, "The Google File System", ACM SIGOPS operating system review, vol. 37, no. 5, pp. 29-43, 2003.

R. Gupta, H. Gupta, U. Nambiar, M. Mohania, "Efficiently querying archived data using hadoop", Proceedings of the 19th ACM international conference on Information and knowledge management, pp. 1301-1304, 2010.

J. Huang, X. Liang, X. Qin, P. Xie, C. Xie, "Scale-rs: An efficient scaling scheme for rs-coded storage clusters", Parallel and Distributed Systems IEEE Transactions on, vol. 26, no. 6, pp. 1704-1717, 2015.

J. Huang, Y. Wang, X. Qin, X. Liang, S. Yin, C. Xie, "Exploiting pipelined encoding process to boost erasure-coded data archival", Parallel and Distributed Systems IEEE Transactions on, vol. 26, no. 11, pp. 2984-2996, 2015.

Hsiao Hung-Chang, Chung Hsueh-Yi, Haiying Shen, chao Yu-Chang, "Load Rebalancing for distributed file system in clouds", IEEE Journal of Transactions on parallel and distributed systems, vol. 24, no. 5, pp. 951-962, 2013.

R.T Kaushik, K Nahrstedt, "T: a data-centric cooling energy costs reduction approach for big data analytics cloud" in Proceedings of the International Conference on High Performance Computing Networking Storage and Analysis., IEEE Computer Society Press, pp. 52, 2012.

R.T Kaushik, K Nahrstedt, "T: a data-centric cooling energy costs reduction approach for big data analytics cloud", Proceedings of the International Conference on High Performance Computing Networking Storage and Analysis., pp. 52, 2012.

L Pamies-Juarez, A Datta, F Oggier, "Rapidraid: Pipelined erasure codes for fast data archival in distributed storage systems", INFOCOM, pp. 1294-1302, 2013, 2013.

L Pamies-Juarez, F Oggier, A Datta, "Decentralized erasure coding for efficient data archival in distributed storage systems" in Distributed Computing and Networking., Springer, pp. 42-56, 2013.